



Centro académico de Limón

Escuela de Computación Lenguajes de programación

Proyecto Programado #4

Videojuego: Colonias Galácticas

Código de curso: IC4700

Estudiante:

Alice Arias Salazar | 2023104639

Heldyis Agüero Espinoza | 2023296812

Yeremi Calvo Porras | 2023083332

Profesor:

Allan Rodríguez Davila

Grupo: 60

I Semestre 2026

Índice

1	Manual de usuario	3
1.1	Abrir el proyecto en VSCode y ejecutar la aplicación	3
1.2	Menu Principal	4
1.3	Rankings	5
1.4	Crear Partida	5
1.5	Unirse a partida.....	7
1.6	Pantalla de juegos	8
1.7	Menú de construcción.....	9
1.8	Menú de Enviar Flota	10
1.9	Estadísticas finales de partida.....	11
2	Descripción del problema.....	12
3	Diseño del programa.....	13
3.1	Diagrama de paquetes.....	13
3.2	Diagrama de comunicación	14
3.3	Diagrama de Distribución.....	15
4	Librerías usadas	15
4.1	React	15
4.2	Express	15
4.3	Socket.IO	15
4.4	Github Pages.....	16
5	Análisis de resultados	16
6	Descripción de algoritmos	19
6.1	Crear partida	19
6.2	Unirse a partida.....	20
6.3	Conquistar.....	20
6.4	Victoria	21
7	Links públicos	22

1 Manual de usuario

Primero deben clonar el repositorio desde GitHub usando GitHub Desktop o directamente en el sitio web del servicio: <https://github.com/Alice-Arias/PY3-Operaci-n-Atlas.git>

Esto descargará todos los archivos en una carpeta local en su computadora.

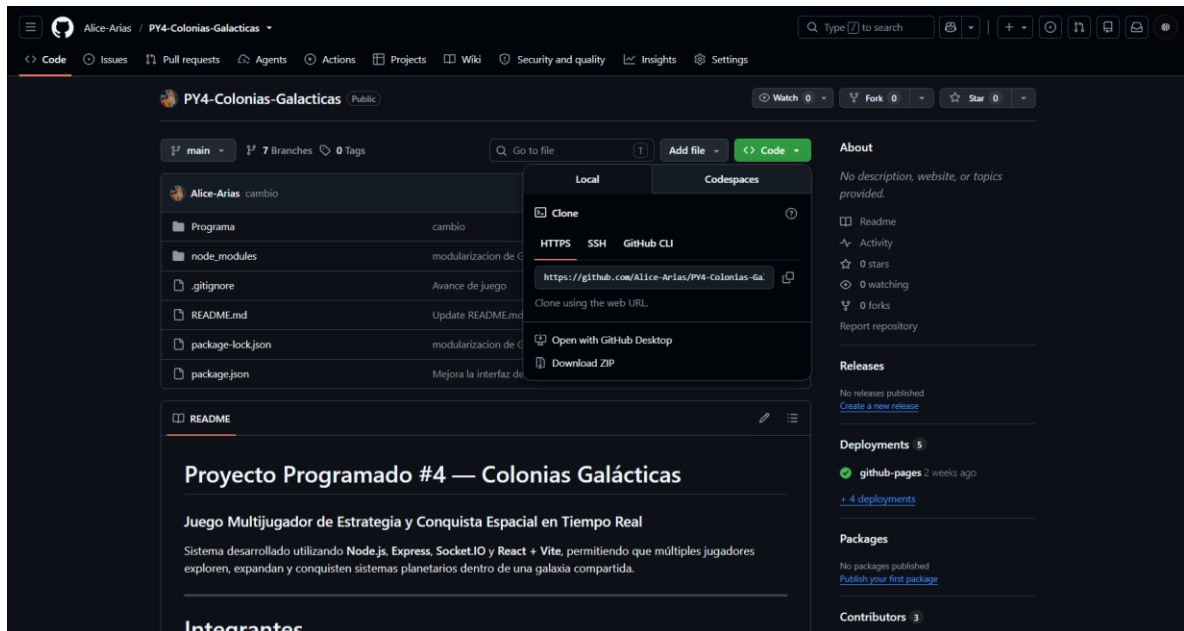


Figura 1

Si prefiere hacerlo de otra manera, puedes ingresar al repositorio y hacer clic en Code. Luego, como se muestra en la Figura 1, selecciona la opción Download ZIP. Una vez que la descarga se complete, descomprime el archivo y estarás listo para continuar con el siguiente paso.

1.1 Abrir el proyecto en VSCode y ejecutar la aplicación

Una vez descargado, abran Visual Studio Code y carguen la carpeta del proyecto vamos a abrir una terminal nueva, esto para comenzar a compilar el proyecto. Para ello, debe colocarse en el fichero “*backend*” y ejecutar el comando “`npm start`”. En otra terminal paralela debe colocarse en el fichero “*frontend*” y ejecutar el comando “`npm run dev`”

```
hedy@hedyisAE MINGW64 ~/Desktop/P4_Lenguajes/programa/frontend (main)
$ npm run dev

> frontend@0.0.0 dev
> vite

1:05:07 p. m. [vite] (client) Re-optimizing dependencies because lockfile has changed

VITE v8.0.16 ready in 1006 ms

→ Local: http://localhost:5173/PY4-Colonias-Galacticas/
→ Network: use --host to expose
→ press h + enter to show help

1:18:56 p. m. [vite] server restarted.
```

Figura 2

```
hedy@hedyisAE MINGW64 ~/Desktop/P4_Lenguajes/programa/backend (main)
$ npm start

> backend@1.0.0 start
> node server.js

Servidor multiplayer listo en puerto 3002
```

Figura 3

Esta combinación de comandos hace posible ejecutar la aplicación de forma local en su dispositivo, se vincula el backend, que corresponde a la lógica de negocio del sistema, con el frontend donde se encuentran los componentes visuales que hacen posible la interacción directa y sencilla entre el usuario y el sistema.

Al ejecutar estos dos comandos, se debe presionar el link brindado tras la ejecución de “npm run dev”, esto abrirá una ventana en su navegador asignado que es donde se estará ejecutando y mostrando la aplicación.

1.2 Menu Principal

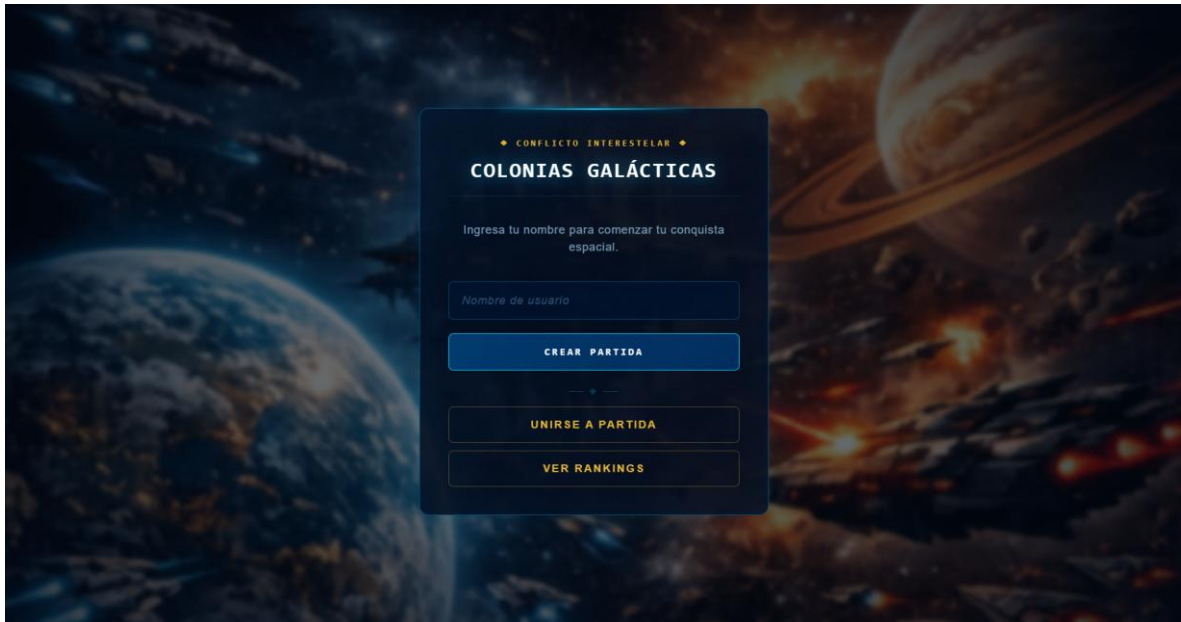


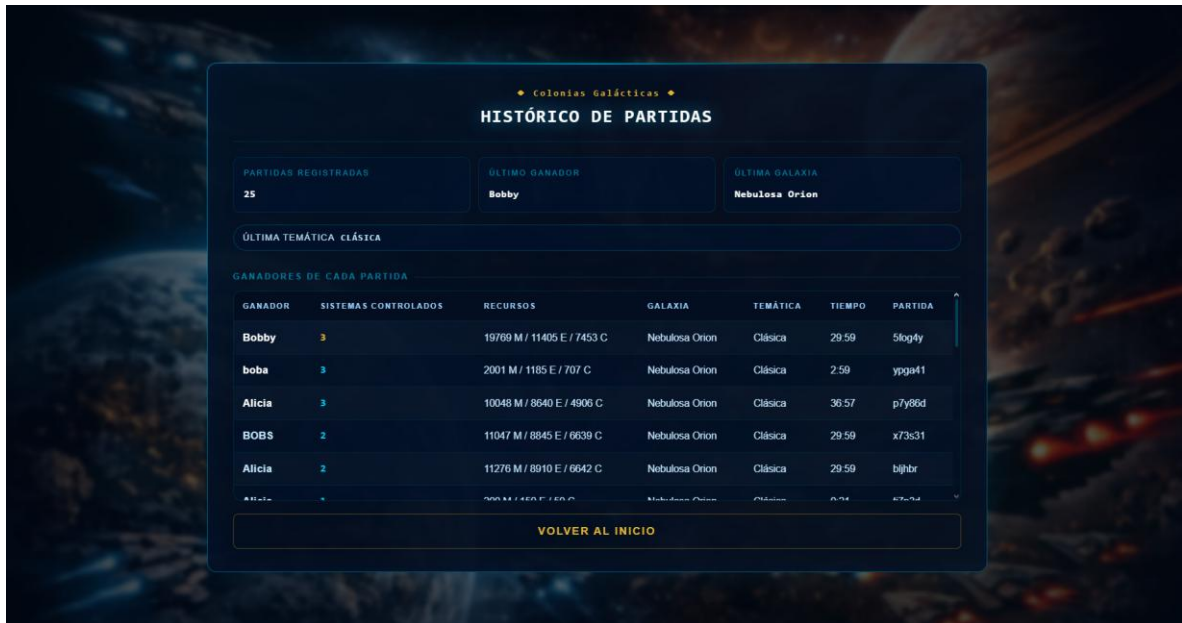
Figura 4

Esta es la primera página que verá el usuario luego de ejecutar el sistema, lo que se muestra es el título del videojuego acompañado de una tarjeta de opciones. Esta tarjeta de opciones cuenta con 4 componentes visuales: Un input, un botón para crear la partida, otro botón para unirse a una partida y otro botón más que les permite a los usuarios visualizar los rankings históricos del videojuego.

El usuario notará que al presionar los botones sin haber escrito un nickname se le presentará un alert en la parte superior de la pantalla, esto se debe a que para crear partida y unirse a una partida es estrictamente necesario contar con un nickname, por otro lado, el usuario sí puede ingresar a la sección de rankings sin haber escrito un nickname previamente.

El nombre de usuario (nickname) debe tener una longitud de más de 3 caracteres, de no cumplirse esta condición se le presentará al usuario otro alert informando al usuario de esto.

1.3 Rankings



Colonias Galácticas						
HISTÓRICO DE PARTIDAS						
PARTIDAS REGISTRADAS	ÚLTIMO GANADOR	ÚLTIMA GALAXIA				
25	Bobby	Nebulosa Orion				
ÚLTIMA TEMÁTICA: CLÁSICA						
GANADORES DE CADA PARTIDA						
GANADOR	SISTEMAS CONTROLADOS	RECURSOS	GALAXIA	TEMÁTICA	TIEMPO	PARTIDA
Bobby	3	19789 M / 11405 E / 7453 C	Nebulosa Orion	Clásica	29:59	5log4y
boba	3	2001 M / 1185 E / 707 C	Nebulosa Orion	Clásica	2:59	ypga41
Alicia	3	10048 M / 8640 E / 4906 C	Nebulosa Orion	Clásica	36:57	p7y86d
BOBS	2	11047 M / 8845 E / 6639 C	Nebulosa Orion	Clásica	29:59	x73s31
Alicia	2	11276 M / 8910 E / 6642 C	Nebulosa Orion	Clásica	29:59	bijbr
VOLVER AL INICIO						

Figura 5

Cuando el usuario presione el botón de rankings del menú principal, se le mostrará la tabla de la imagen anterior. En esta tabla el usuario puede visualizar los registros históricos de puntuación y resultados de las partidas que se han jugado. En la tabla el usuario puede visualizar el nombre de los jugadores, la cantidad de sistemas controlados por cada uno, los recursos con los que finalizó la partida, la galaxia en la que se desarrolló la partida, la temática, el tiempo de la partida y el código de la partida.

En la parte inferior del componente el usuario puede encontrar un botón para volver la pantalla de inicio, al presionarla se le dirigirá de regreso al menú principal.

1.4 Crear Partida



Colonias Galácticas		
NUEVA PARTIDA		
VISTA PREVIA CLÁSICA		
IDENTIFICACIÓN		
NOMBRE DE LA PARTIDA		
Santurion		
CONFIGURACIÓN ESPACIAL		
GALAXIA		
Andromeda Prime		
TEMÁTICA DEL JUEGO		
Clásica		
La temática cambia la ambientación sin impedir jugar. La partida termina por tiempo o cuando un jugador controla el porcentaje fijado de la galaxia.		
MÁX. JUGADORES		
2		
PARÁMETROS DE COMBATE		
DURACIÓN REAL DE LA PARTIDA (SEG.)		
1800		
Este tiempo es el que importa dentro del juego.		
TIEMPO DE ESPERA DEL LOBBY (SEG.)		
60		
Solo controla cuánto tiempo hay para reunir jugadores.		
CONTROL PARA GANAR (%)		
70		
NIVEL DE RECURSOS		
Alto		
DOTACIÓN INICIAL		
RECURSOS ASIGNADOS		
Minerales 600		
Energía 250		
Cristales 100		
CREAR PARTIDA		
VOLVER AL INICIO		

Figura 6

Cuando se presiona el botón de crear partida se despliega la interfaz mostrada en la imagen anterior, esta interfaz ofrece diversas opciones de personalización y configuración de la partida, entre ellos:

- Identificación: Nombre de la partida
- Configuración espacial: Galaxia a seleccionar dentro de las cargadas en el archivo json, temática del juego y número máximo de jugadores
- Parámetros de combate: Duración de la partida en segundos, tiempo de espera del lobby, porcentaje de control para ganar y nivel de recursos.

Finalmente, en la parte derecha del panel se muestra la sección de dotación inicial, ese cuadro se muestran los recursos iniciales de cada jugador. Los recursos iniciales cambian en función del nivel de recursos seleccionado por el usuario en tiempo real, esto le permite al usuario un mejor control sobre su configuración. Además, ahí se encuentran los botones de crear partida y volver al inicio. Si el usuario presiona el último mencionado, se le devolverá a la pantalla del menú principal. Por el contrario, cuando se cree la partida el usuario será movido a un lobby de espera donde los demás jugadores podrán ir uniéndose.

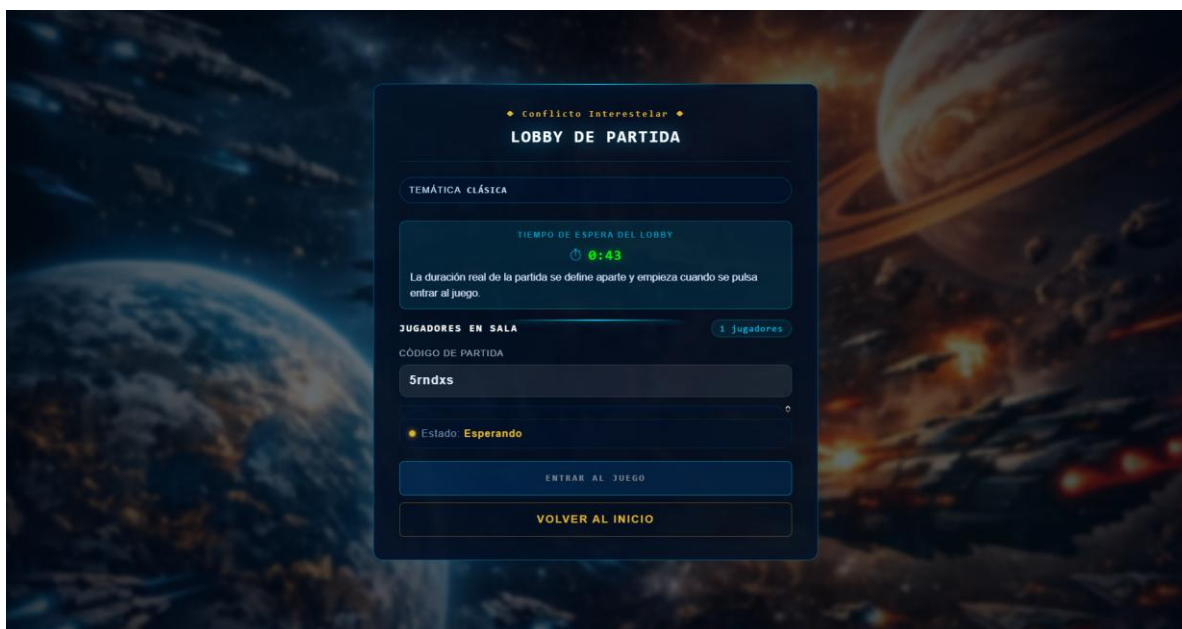


Figura 7

Esa corresponde a la pantalla del lobby a la que son dirigidos los jugadores desde crear partida y desde el menú de unirse a una partida.

En esta pantalla se le muestra al usuario el tiempo restante de espera, la temática del juego en la parte superior, la cantidad de jugadores en el lobby, el código de la partida con el que los jugadores podrán unirse a la misma y una lista que se desplegará conforme empiecen a ingresar jugadores al lobby.

En la parte inferior se le mostrará al jugador “dueño” del lobby el botón para iniciar la partida, tanto el creador de la partida como los demás jugadores tendrán disponible el botón para volver al inicio.

1.5 Unirse a partida

Cuando un usuario ingrese en esta sección se le mostrarán las partidas disponibles en el sistema, es por esto por lo que hay 2 vistas posibles.

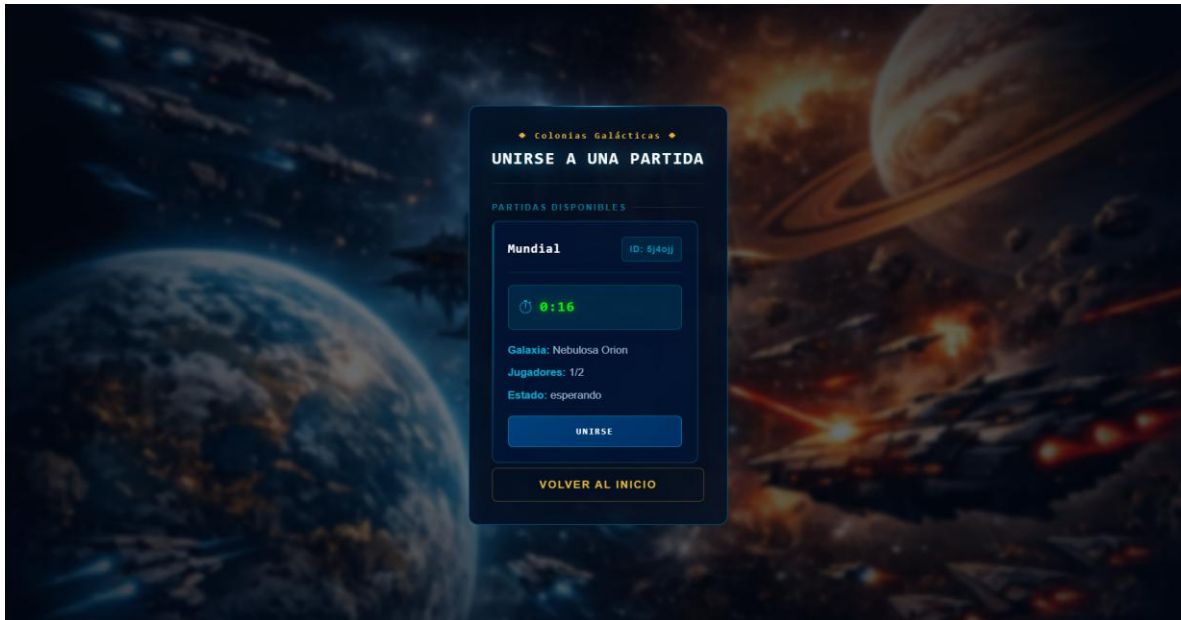


Figura 8

La vista anterior corresponde a cuando hay una partida creada, el usuario puede visualizar el tiempo restante del lobby, el nombre de la partida, la cantidad de jugadores en el lobby, la galaxia de la partida y el estado del lobby. Si el usuario lo desea puede presionar el botón de unirse, esto lo llevará a la pantalla de lobby previamente explicada.

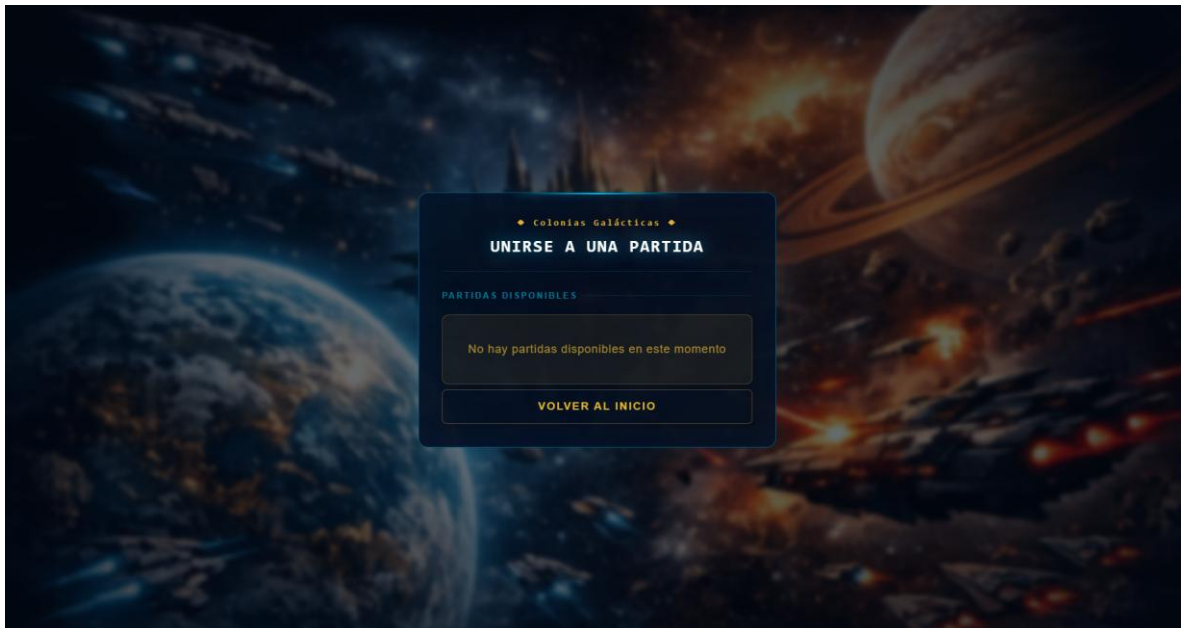


Figura 9

Esta es la pantalla que se le mostrará al usuario en caso de que no hayan partidas.

1.6 Pantalla de juegos

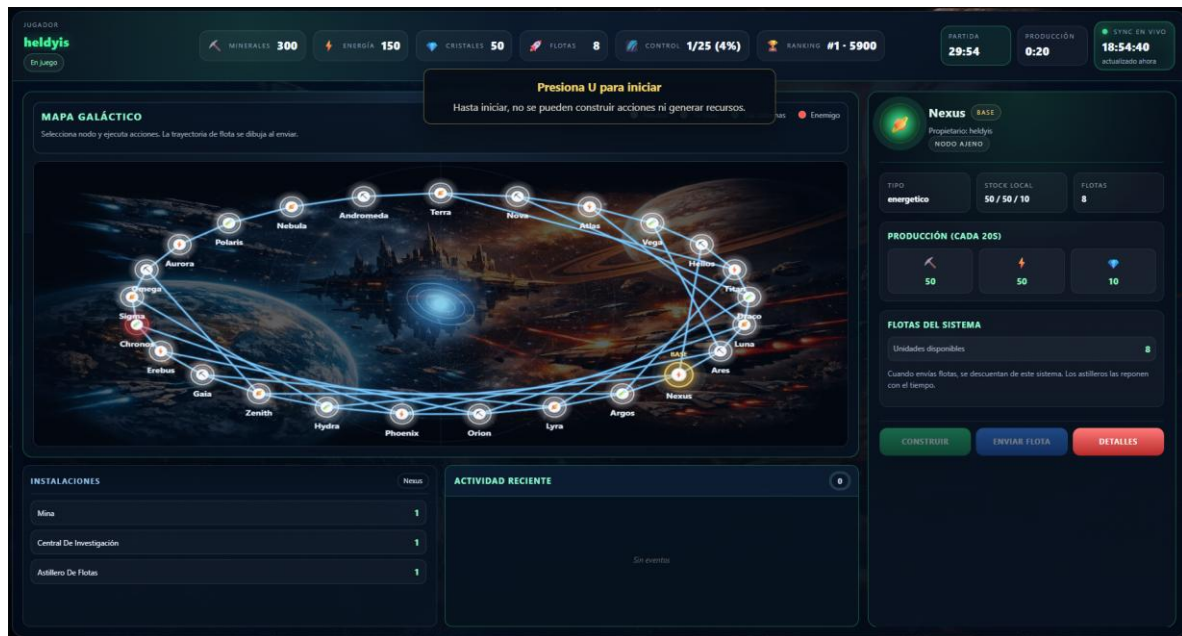


Figura 10

Cuando el jugador dueño del lobby presione el botón para iniciar la partida, se le mostrará esta pantalla de juego.

Dentro, antes de iniciar, cualquiera de los jugadores debe presionar la tecla U para iniciar la partida.

En la parte superior de la pantalla se despliega la barra de estado global del jugador, donde se identifica al usuario con el nickname ingreso previamente y se muestran los recursos disponibles en el imperio, que incluyen 300 unidades de minerales, 150 de energía y 50 de cristales, según la configuración del jugador creador de la partida. Junto a estos contadores, el sistema detalla que el jugador posee un total de 8 flotas globales y controla 1 de los 25 nodos de la galaxia, lo que equivale al 4% del mapa, situándolo temporalmente en el primer puesto del ranking con 5900 puntos. Al costado derecho de esta barra, varios cronómetros monitorizan el progreso de la partida, registrando un tiempo de juego, una cuenta regresiva de 20 segundos para el próximo ciclo de producción y un reloj que sincroniza la hora del servidor en vivo.

El centro de la interfaz está dominado por el Mapa Galáctico, una representación visual de una red interconectada de 25 nodos estelares, este caso según la galaxia seleccionada en el menú de configuración, distribuidos de forma elíptica alrededor de un núcleo espacial. Cada nodo posee un nombre único, como *Andromeda*, *Polaris* o *Nexus*, y están unidos por líneas de color azul brillante que trazan las rutas hiperespaciales válidas para el desplazamiento de las flotas. Una leyenda en la esquina superior derecha del mapa ayuda a distinguir los sistemas propios de los enemigos mediante un código de colores.

Al seleccionar un nodo en el mapa, en este caso la base llamada Nexus, el panel lateral derecho se activa para desglosar sus especificaciones. Este apartado indica que se trata de un planeta de tipo energético y muestra un stock local de 50/50/10, además de las 8 flotas que

se encuentran estacionadas allí. También detalla la producción específica del nodo cada 20 segundos —aportando 50 de minerales, 50 de energía y 10 de cristales— e incluye una advertencia que explica que el envío de flotas descontará unidades de este sistema, las cuales se repondrán con el tiempo gracias a los astilleros. En la parte inferior de este panel, tres botones de colores permiten al jugador ejecutar acciones directas: Construir (en verde) para levantar infraestructuras, Enviar Flota (en azul) para movilizar tropas y Detalles (en rojo) para abrir una vista avanzada.

Finalmente, la sección inferior de la pantalla se divide en dos bloques operativos. A la izquierda se ubica el panel de Instalaciones, que lista los edificios construidos en el nodo seleccionado; actualmente, Nexus cuenta con una Mina, una Central de Investigación y un Astillero de Flotas, todos ellos en el nivel 1. A la derecha se encuentra el panel de Actividad Reciente, un registro cronológico diseñado para mostrar los eventos, combates y movimientos del juego, el cual permanece vacío y con el indicador *"Sin eventos"* cuando no hay actividad registrada en el turno actual.

1.7 Menú de construcción

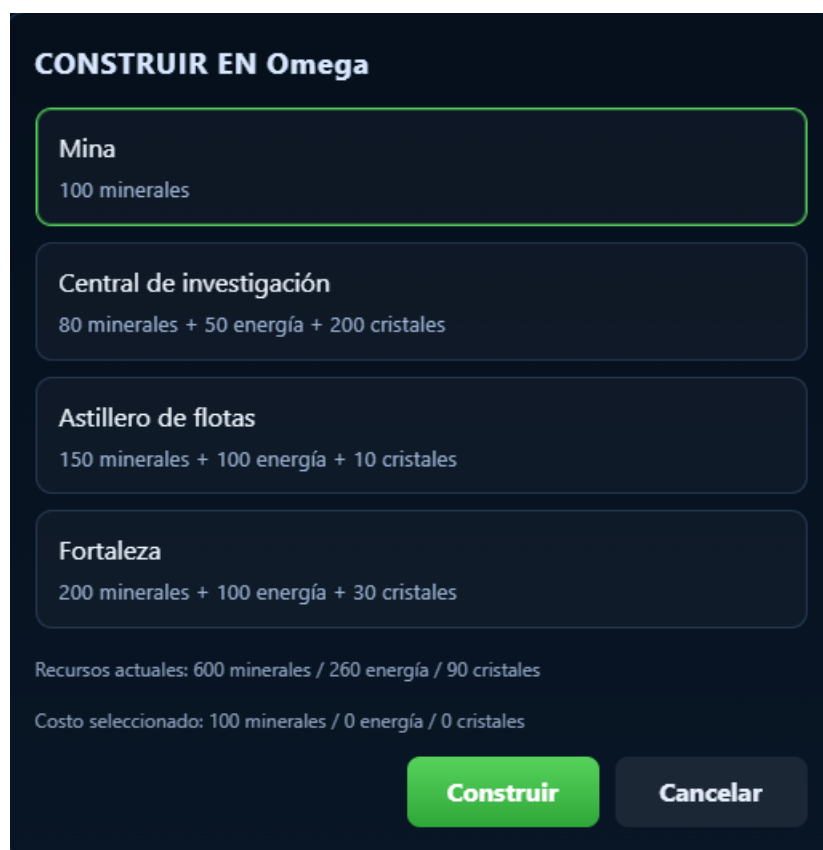


Figura 11

Al presionar el botón de construcción se muestra esta interfaz que presenta una lista con cuatro opciones de construcción disponibles, detallando el coste específico de cada una en minerales, energía y cristales: una Mina, una Central de investigación, un Astillero de flotas y una Fortaleza. En la parte inferior del panel se ofrece un balance financiero en tiempo real, desglosando los Recursos actuales del jugador justo por encima del Costo seleccionado para

la estructura activa. Finalmente, la acción se completa mediante dos botones principales: un botón verde para Construir y confirmar la orden, y un botón oscuro para Cancelar y cerrar el menú.

1.8 Menú de Enviar Flota

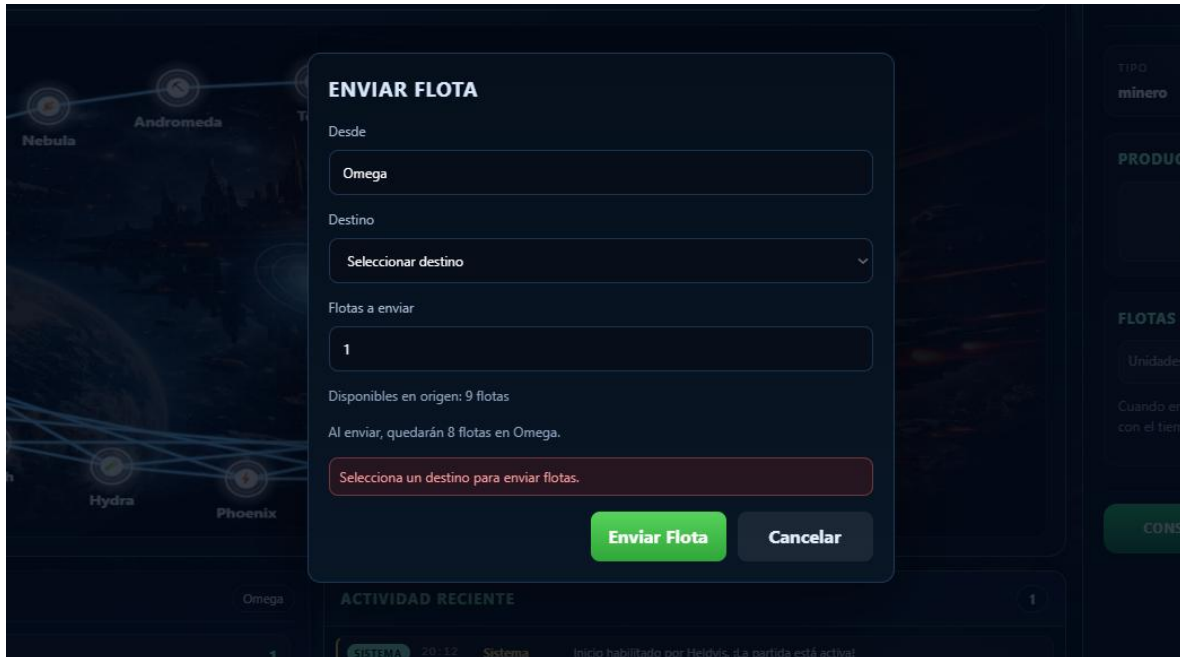


Figura 12

En el menú de enviar flota, que se presenta tras presionar el botón de enviar flota, se presentan 3 campos de texto:

- Desde: Indica el punto de origen de la orden, fijado actualmente en el nodo Omega.
- Destino: Un menú desplegable que requiere que el usuario elija el sistema receptor.
- Flotas a enviar: Un campo numérico para introducir la cantidad de naves a movilizar.

Bajo estos campos, la interfaz calcula dinámicamente el impacto del movimiento, informando que hay 9 flotas disponibles en el origen y que, tras el envío, quedarán 8 flotas en Omega. Debido a que aún no se ha elegido un sistema de llegada, se muestra un cuadro de advertencia de color rojo con el mensaje *"Selecciona un destino para enviar flotas"*. La operación se puede confirmar con el botón verde Enviar Flota o abortar mediante el botón Cancelar.

1.9 Estadísticas finales de partida



Figura 13

Esta es la vista de estadísticas finales que se muestra al finalizar el juego. En la parte superior, el menú organiza los datos generales de la sesión en cinco bloques de información. Aquí se identifica el código único de la partida, la galaxia en la que se combatió, un tiempo de juego total, y una configuración de temática. Asimismo, se destaca formalmente que el ganador de la sesión, una victoria que también se enfatiza visualmente en la zona inferior de la pantalla mediante un rótulo flotante con el texto "*Ganador*" acompañado del nombre del usuario.

Justo debajo de estos datos, una nota aclara al usuario que el orden de las posiciones y el puntaje final se calculan en función de los sistemas controlados, los recursos acumulados y las estructuras construidas. Para salir de este informe, la interfaz dispone en su esquina inferior derecha de un botón gris para "Ver última partida" y un botón verde para "Cerrar" la ventana.

2 Descripción del problema

Los videojuegos de estrategia parecidos al implementado en este proyecto requieren una administración eficiente de recursos expansión territorial, movilización de unidades e interacción simultánea entre varios jugadores dentro de un universo compartido. Para lograr una experiencia fluida, es necesario contar con mecanismos que permitan la sincronización en tiempo real de las acciones realizadas por cada participante del juego, esto para garantizar que todos los presentes observen los mismos eventos y cambios en el estado del juego.

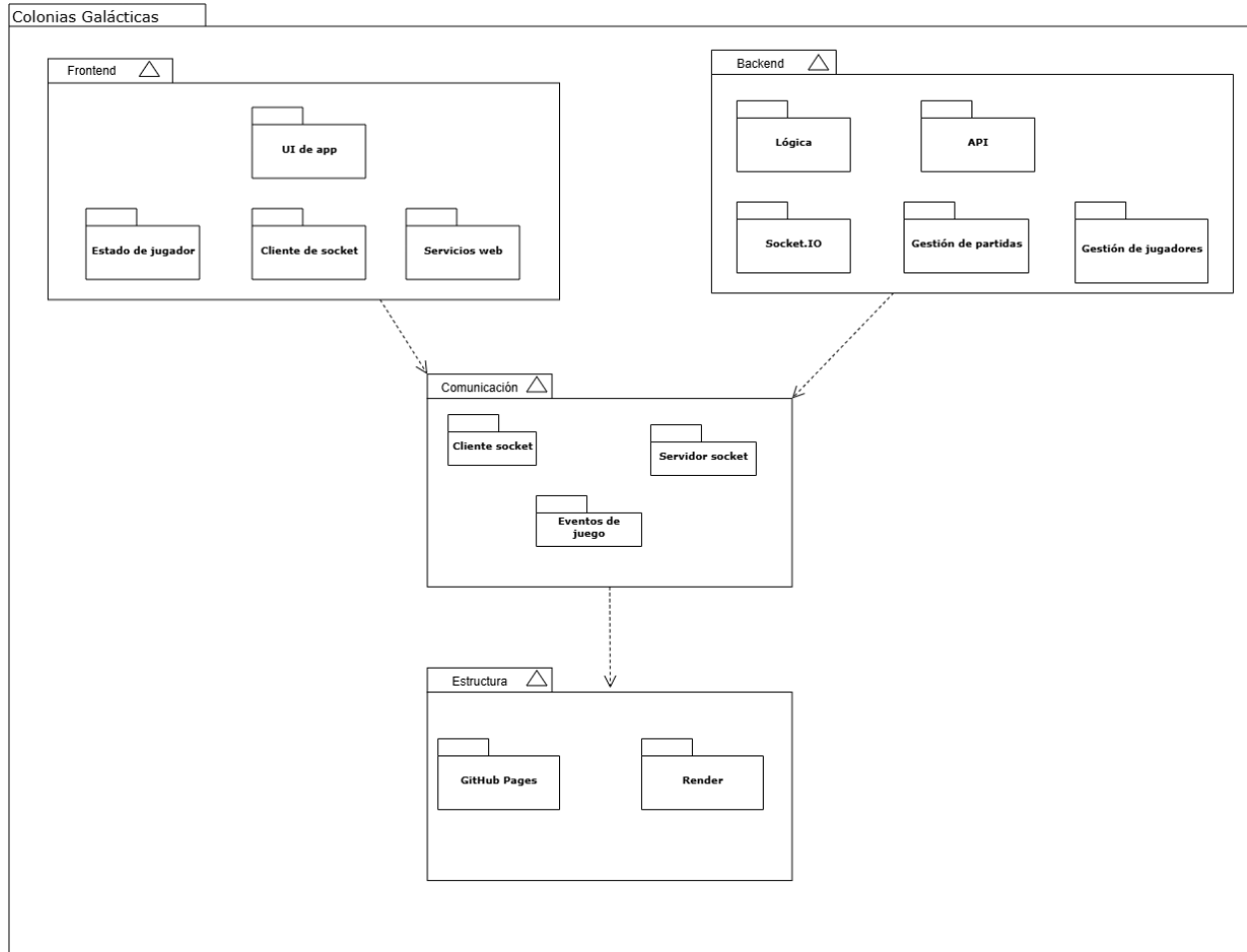
El principal problema de este proyecto consiste en diseñar y desarrollar un videojuego web que permita la creación y gestión de partidas de estrategia espacial en línea. Este sistema debe representar el universo mediante un grafo compuesto por varios planetas conectados a través de rutas espaciales donde los jugadores puedan explorar, controlar, producir recursos, construir, movilizar flotas y conquistar sistemas pertenecientes a otros participantes.

Además, la solución debe soportar múltiples usuarios conectados simultáneamente, administrar la concurrencia de las acciones realizadas en el juego y mantener la consistencia de la información compartida. Para esto, se requiere la implementación de una arquitectura cliente-servidor capaz de procesar la lógica del juego en el servidor y ofrecer una GUI en el cliente, esto permite la comunicación en tiempo real mediante tecnologías modernas como WebSockets.

Finalmente, el sistema debe proporcionar mecanismos para la creación y configuración de partidas, visualización de estadísticas, generación de rankings históricos y almacenamiento persistente de la información, garantizando una experiencia competitiva, dinámica y escalable para todos los jugadores.

3 Diseño del programa

3.1 Diagrama de paquetes



3.2 Diagrama de comunicación

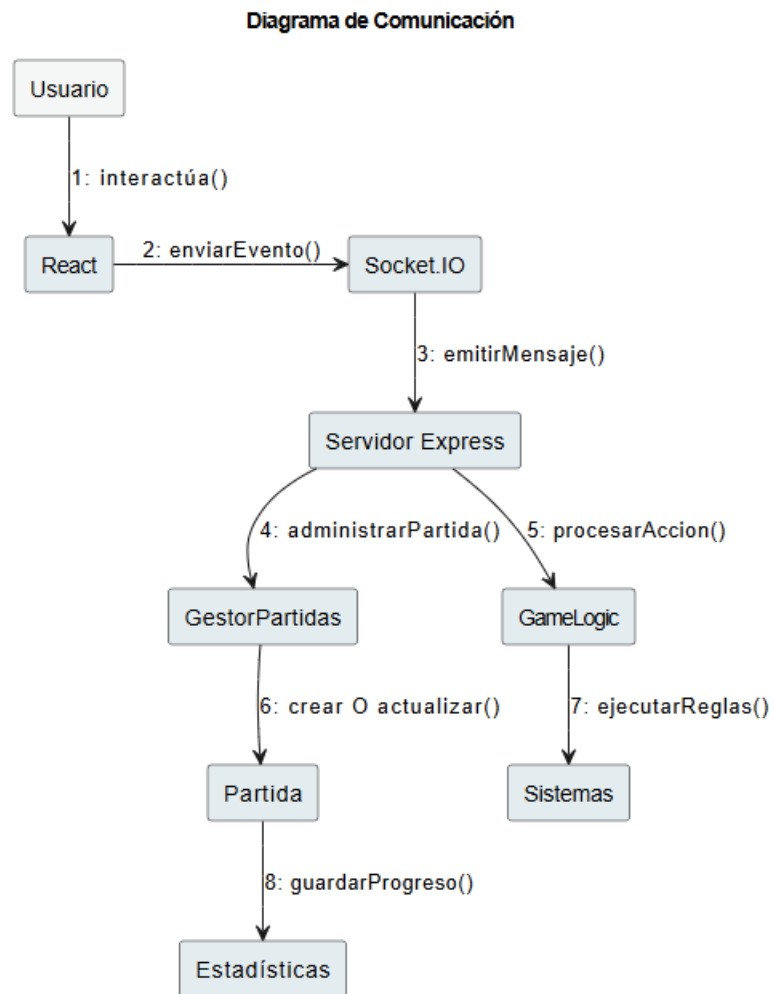


Figura 14

3.3 Diagrama de Distribución

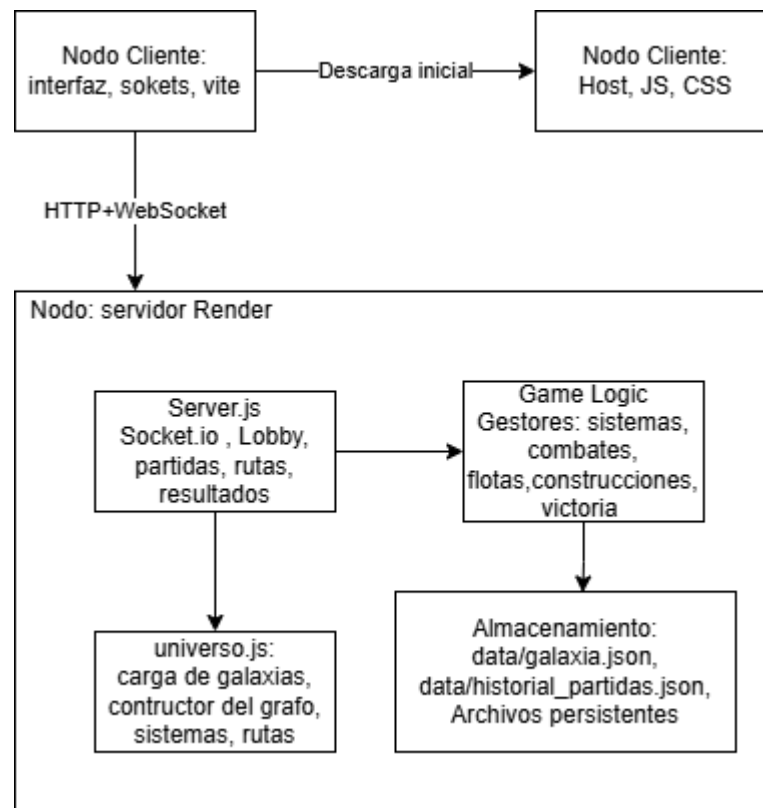


Figura 15

4 Librerías usadas

4.1 React

React fue utilizado para el desarrollo del frontend de la aplicación. Este framework basado en componentes permitió construir una interfaz de usuario dinámica, modular y reutilizable. Gracias a React fue posible gestionar el estado de la aplicación, actualizar la información mostrada en pantalla de manera eficiente y facilitar la navegación entre las diferentes vistas del sistema, como la autenticación, creación de partidas, sala de espera, ranking y la página de juego, que es donde se desarrolla el gameplay.

4.2 Express

Express fue utilizado para el desarrollo del servidor backend. Esta librería permitió crear una API web encargada de gestionar la lógica del juego, el manejo de partidas, la administración de jugadores y la comunicación con el frontend. Además, facilitó la definición de rutas y el procesamiento de solicitudes HTTP de manera sencilla y organizada.

4.3 Socket.IO

Socket.IO fue utilizado para implementar la comunicación en tiempo real entre los clientes y el servidor. Esta tecnología permitió sincronizar las acciones de todos los jugadores conectados a una partida, incluyendo la creación y unión a partidas, el inicio del juego, los

movimientos realizados por los jugadores y la actualización inmediata de la información compartida. Su uso fue fundamental para garantizar una experiencia multijugador interactiva y concurrente.

4.4 Github Pages

GitHub Pages fue utilizado para publicar y alojar el frontend de la aplicación. Gracias a esta plataforma fue posible desplegar la interfaz web y hacerla accesible mediante una dirección pública, facilitando las pruebas y la interacción con el sistema sin necesidad de instalaciones adicionales por parte de los usuarios.

5 Análisis de resultados

Objetivo	Alcanzado	No alcanzado
Desarrollar una aplicación web multijugador, basada en un juego de estrategia y conquista espacial.	X	
Implementar una arquitectura cliente-servidor utilizando tecnologías web modernas.	X	
Permitir la creación y administración de partidas configurables por los usuarios.	X	
Implementar autenticación mediante nickname para el acceso al sistema.	X	
Cargar y representar galaxias mediante grafos contruidos a partir de archivos JSON.	X	

Objetivo	Alcanzado	No alcanzado
Gestionar sistemas planetarios con recursos, instalaciones, flotas y propietarios.	X	
Permitir que múltiples jugadores se conecten simultáneamente a una misma partida.	X	
Implementar salas de espera y mecanismos de inicio sincronizado de partidas.	X	
Desarrollar un mapa galáctico interactivo para la visualización del universo de juego.	X	
Implementar la generación automática y gestión de recursos (minerales, energía y cristales).	X	
Permitir la construcción de infraestructuras dentro de los sistemas controlados.	X	
Implementar la movilización de flotas respetando las conexiones definidas en el grafo.	X	

Objetivo	Alcanzado	No alcanzado
Desarrollar un sistema automático de conquista y resolución de combates.	X	
Sincronizar en tiempo real las acciones y eventos del juego mediante WebSockets.	X	
Reflejar en tiempo real cambios de propietarios, construcciones, movimientos y combates.	X	
Implementar condiciones de finalización de partida configurables.	X	
Generar estadísticas detalladas de cada partida finalizada.	X	
Mantener un ranking histórico de ganadores y resultados de partidas.	X	
Gestionar la persistencia de la información mediante archivos y/o base de datos.	X	

Objetivo	Alcanzado	No alcanzado
Aplicar principios de programación orientada a objetos en el desarrollo del proyecto.	X	
Utilizar control de versiones mediante GitHub para la gestión colaborativa del proyecto	X	
Proporcionar una interfaz web intuitiva y funcional para los jugadores.	X	
Garantizar la concurrencia y experiencia multiusuario en tiempo real.	X	
Documentar adecuadamente el diseño, funcionamiento y algoritmos principales del sistema.	X	

6 Descripción de algoritmos

6.1 Crear partida

Este algoritmo se encarga de generar una nueva partida dentro del sistema cuando un usuario selecciona la opción de crear juego. Su objetivo es inicializar todos los elementos necesarios para que otros jugadores puedan unirse posteriormente y comenzar una sesión multijugador.

El proceso inicia recibiendo la configuración seleccionada por el usuario, como el nombre de la partida, la galaxia que se utilizará, la cantidad máxima de jugadores, el tiempo de espera y los recursos iniciales. A partir de esta información se genera un identificador único que permitirá distinguir la partida de las demás existentes en el servidor.

Posteriormente, el algoritmo construye el universo de juego cargando la galaxia seleccionada y prepara la temática visual correspondiente. Con estos datos se crea una nueva instancia de partida que almacenará toda la información necesaria para el desarrollo de la sesión.

Una vez creada la partida, el usuario que la generó es registrado automáticamente como anfitrión y se agrega como primer jugador. Después, la partida se almacena en la colección de partidas activas del servidor para que pueda ser consultada por otros usuarios.

Finalmente, el algoritmo conecta al anfitrión con la sala de comunicación correspondiente y notifica tanto al creador como a los posibles participantes sobre la existencia de la nueva partida mediante eventos en tiempo real, esto como lo vimos anteriormente se visualiza en la sección de unirse a partida. De esta forma, la sala de espera queda disponible para que otros jugadores puedan unirse antes del inicio del juego.

6.2 Unirse a partida

Este algoritmo permite que un jugador se incorpore a una partida existente dentro del sistema. Su objetivo es gestionar el ingreso de nuevos participantes y garantizar que la información de la sala de juego permanezca consistente para todos los usuarios conectados.

El proceso inicia cuando un jugador selecciona una partida disponible. El algoritmo verifica que la partida exista y que todavía tenga espacio para nuevos participantes. Si la capacidad máxima ya fue alcanzada, se le indica al usuario que no es posible unirse.

Posteriormente, se comprueba si el jugador ya formaba parte de la partida. Esta validación permite evitar registros duplicados y también facilita la recuperación de la sesión en caso de desconexiones accidentales. Si el jugador ya existía, el sistema actualiza su estado para permitirle continuar jugando sin perder su progreso previo.

Finalmente, el algoritmo sincroniza la información de la partida con el jugador que acaba de ingresar. Si la partida aún se encuentra en la sala de espera, se actualiza la información para todos los participantes. En caso de que la partida ya haya comenzado, el jugador recibe inmediatamente el estado actual del juego para que pueda continuar interactuando sin afectar el desarrollo de la sesión.

De esta manera, el algoritmo garantiza la correcta incorporación de jugadores, el manejo de reconexiones y la sincronización de la información en tiempo real dentro de las partidas multijugador.

6.3 Conquistar

Para llevar a cabo las conquistas, el sistema cuenta con 2 casos específicos. Cuando un nodo no tiene dueño y el otro caso contrario.

Cuando un jugador envía flotas hacia un sistema que no posee propietario, el algoritmo verifica que tanto el jugador como el sistema existan dentro de la partida. Una vez realizada la validación, el sistema pasa a estar bajo el control del jugador que realizó la acción.

Posteriormente, se actualizan los datos del sistema para registrar al nuevo propietario y la cantidad de flotas que permanecen en él. Además, el sistema se agrega a la lista de territorios controlados por el jugador y se actualiza la cantidad total de sistemas bajo su dominio.

Finalmente, el algoritmo devuelve la información necesaria para notificar al resto de participantes que el sistema ha sido conquistado exitosamente. Este procedimiento representa la forma más sencilla de expansión territorial, ya que no existe oposición por parte de otro jugador.

Por otro lado, cuando un jugador logra conquistar un sistema que previamente pertenecía a otro participante, el algoritmo realiza una serie de actualizaciones adicionales para reflejar el cambio de control.

En primer lugar, se identifica al propietario anterior y se elimina el sistema de la lista de territorios que este controla. También se actualiza su cantidad total de sistemas controlados para reflejar la pérdida del territorio.

Posteriormente, las estructuras defensivas y de producción asociadas al sistema son eliminadas, simulando los efectos de la invasión y evitando que el nuevo propietario herede ventajas estratégicas obtenidas por el dueño anterior.

Una vez completado este proceso, el sistema es asignado al jugador conquistador. Se actualiza la información del propietario, las flotas supervivientes y el estado general del sistema. Si la conquista corresponde a la asignación inicial de una base de jugador al inicio de la partida, también se crean automáticamente las estructuras básicas necesarias para comenzar el desarrollo de la colonia.

Finalmente, el sistema se incorpora a los territorios controlados por el nuevo propietario y se recalculan sus estadísticas de dominio territorial. De esta forma, el algoritmo garantiza que la información de ambos jugadores permanezca consistente después de cada conquista.

6.4 Victoria

Este algoritmo se encarga de determinar si una partida ha llegado a su fin y de identificar al jugador ganador. Su objetivo es verificar continuamente las condiciones de victoria definidas para el juego y finalizar la partida cuando alguna de ellas se cumple.

El proceso inicia comprobando si la partida ya fue finalizada previamente. Si existe un ganador registrado o la partida ya terminó, este proceso se hace simplemente para evitar casos que se salgan de control y evitar así también incongruencias, además, el algoritmo detiene su ejecución para evitar evaluaciones innecesarias.

Posteriormente, se obtiene la lista de jugadores que continúan activos dentro de la partida, excluyendo aquellos que hayan sido eliminados. Con esta información se analizan las diferentes condiciones de victoria establecidas por las reglas del juego.

La primera condición consiste en verificar si algún jugador controla un porcentaje determinado de los sistemas planetarios existentes en la galaxia. Para ello, el algoritmo calcula la cantidad real de sistemas bajo el control de cada jugador y la compara con el porcentaje de victoria configurado para la partida. Si algún participante alcanza o supera dicho valor, es declarado ganador y la partida finaliza inmediatamente.

Si ningún jugador cumple la condición anterior, el algoritmo evalúa una segunda posibilidad: que únicamente permanezca un jugador activo. Esta situación ocurre cuando todos los demás participantes han sido eliminados o han perdido el control de sus territorios. En este caso, el jugador restante es declarado ganador y la partida concluye.

Finalmente, existe una tercera condición de victoria relacionada con el tiempo máximo de duración de la partida. Cuando se alcanza este límite, otro procedimiento especializado calcula los resultados finales y determina la posición de cada participante según los criterios de puntuación establecidos por el juego.

Gracias a este algoritmo, el sistema puede identificar de forma automática cuándo debe finalizar una partida y garantizar que el ganador sea determinado de acuerdo con las reglas definidas para la competencia.

7 Links públicos

Backend: <https://py4-colonias-galacticas-1.onrender.com>

Frontend: <https://alice-arias.github.io/PY4-Colonias-Galacticas/>